

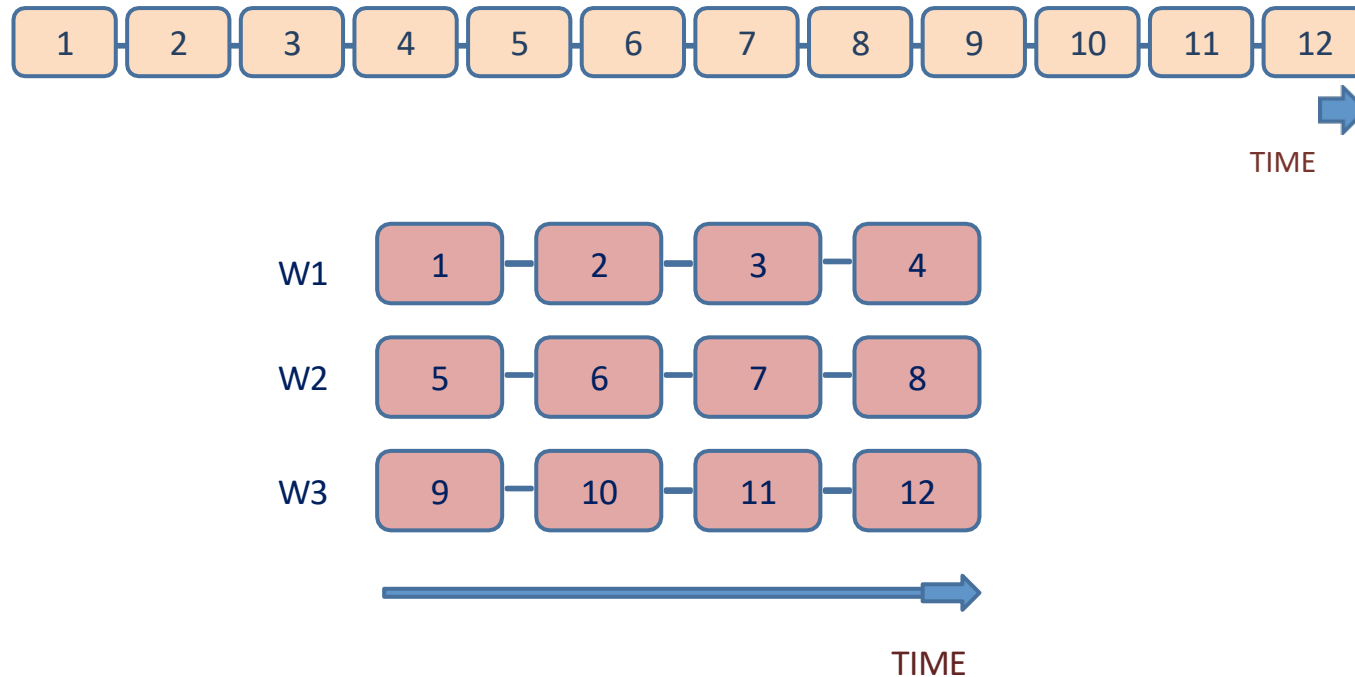
Lecture 3: Speedup, Scalability, Efficiency, Performance

Beakal Gizachew Assefa

Perfect Speedup

- Number of cores/processors= p
- Serial run-time = T_s or T_1 (run-time on 1 core)
- Parallel run-time = T_p (run-time on p cores)
- **Ideal case:** $T_p = T_1 / p$
- Called **perfect** or **linear** speedup (a speedup of p)

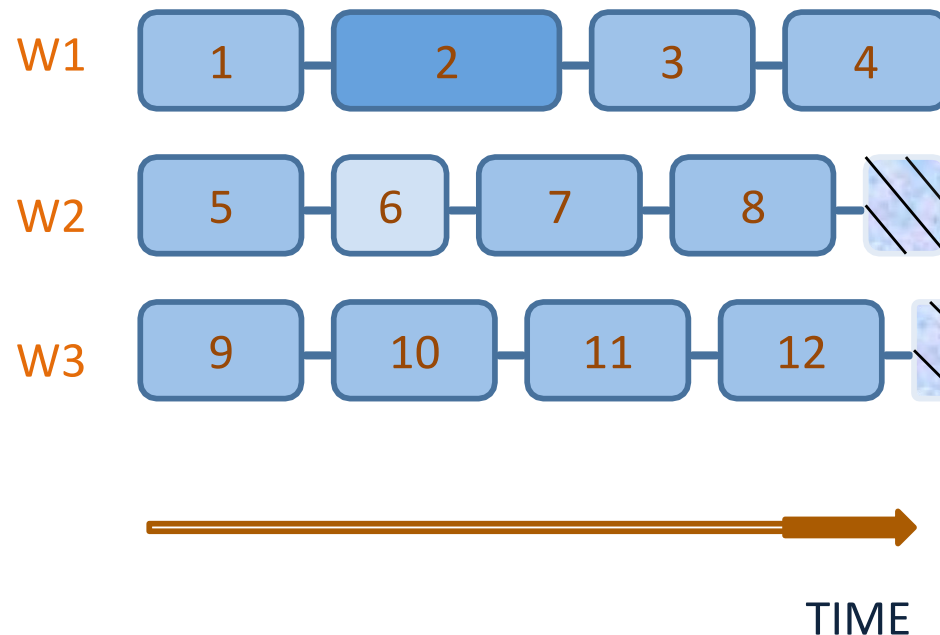
Perfect Speedup



- Ideal case: $T_p = T_1 / p$
- Called **perfect** or **linear** speedup (a speedup of p)
- But this is **rarely** the case
 - Why?

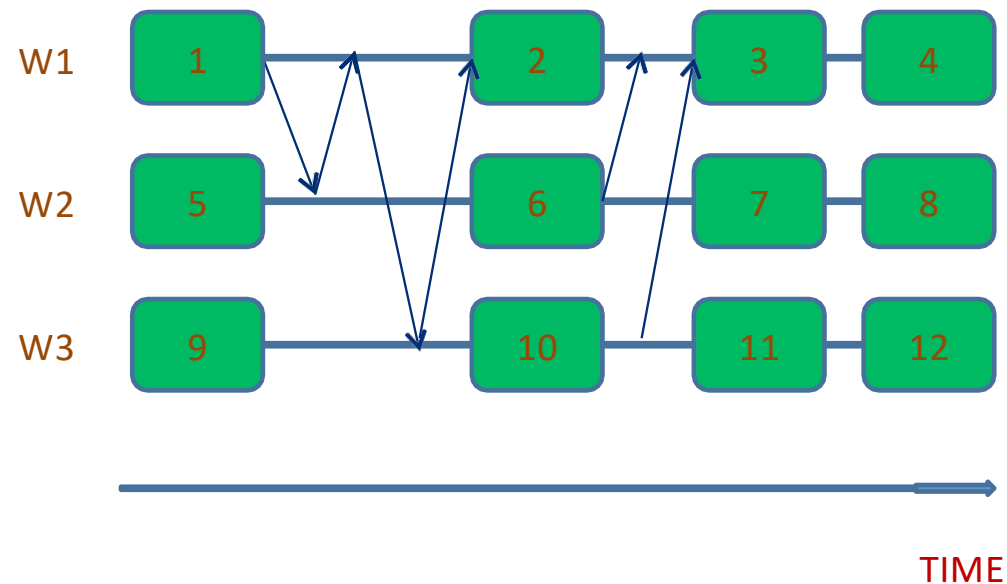
Perfect Speedup is Rarely the Case

- **Because of load imbalance**
 - Some tasks executed by different cores at different speeds
 - Some cores/processors stay idle while waiting for others to finish



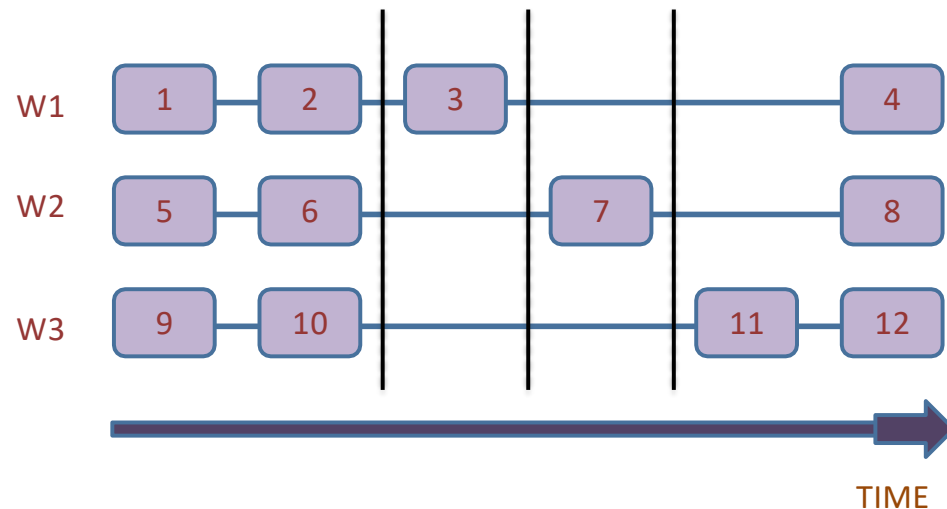
Perfect Speedup is Rarely the Case

- **Because of Communication**
 - Communication processes (arrows represents messages) limit scalability if they cannot be overlapped with each other or with computation



Perfect Speedup is Rarely the Case

- Because of Algorithmic Limitations or Non-parallelizable Sections
 - Operations cannot be done in parallel because of mutual **dependencies**, can only be performed one after another.
- Shared resources **serialises** execution
 - I/O or shared paths to memory in multicore chips



Perfect Speedup is Rarely the Case

- Because of Parallelization Overhead
 - Creating threads or processes is not free, there is always a startup overhead, which cannot be eliminated
- **Superlinear** speedup: also rare, but possible:
 - Typically due to having more cache space and better cache utilization

Speedup & Efficiency

- **Speedup** on p cores: $S_p = T_1 / T_p$
- **Efficiency** on p cores:
 - $E_p = S_p / p$ (note that p represents perfect speedup)
 - $E_p = T_1 / (pT_p)$
- For p cores:
 - $S_{ideal} = p$, $E_{ideal} = 1$ or 100%
- For advertisements, use speedup, not efficiency :)

Amdahl's Law

- Captures one aspect of the potential overheads:
 - **non--parallelizable serial** computations
- $T_1 = W_{\text{ser}} + W_{\text{par}}$
 - W_{ser} : Time spent on non--parallelizable serial work
 - W_{par} : Time spent on parallelizable work
- $T_p \geq W_{\text{ser}} + W_{\text{par}} / p$

Amdahl's Law

- **Amdahl's Law:**
$$S_P \leq \frac{W_{ser} + W_{par}}{W_{ser} + W_{par}/P}$$

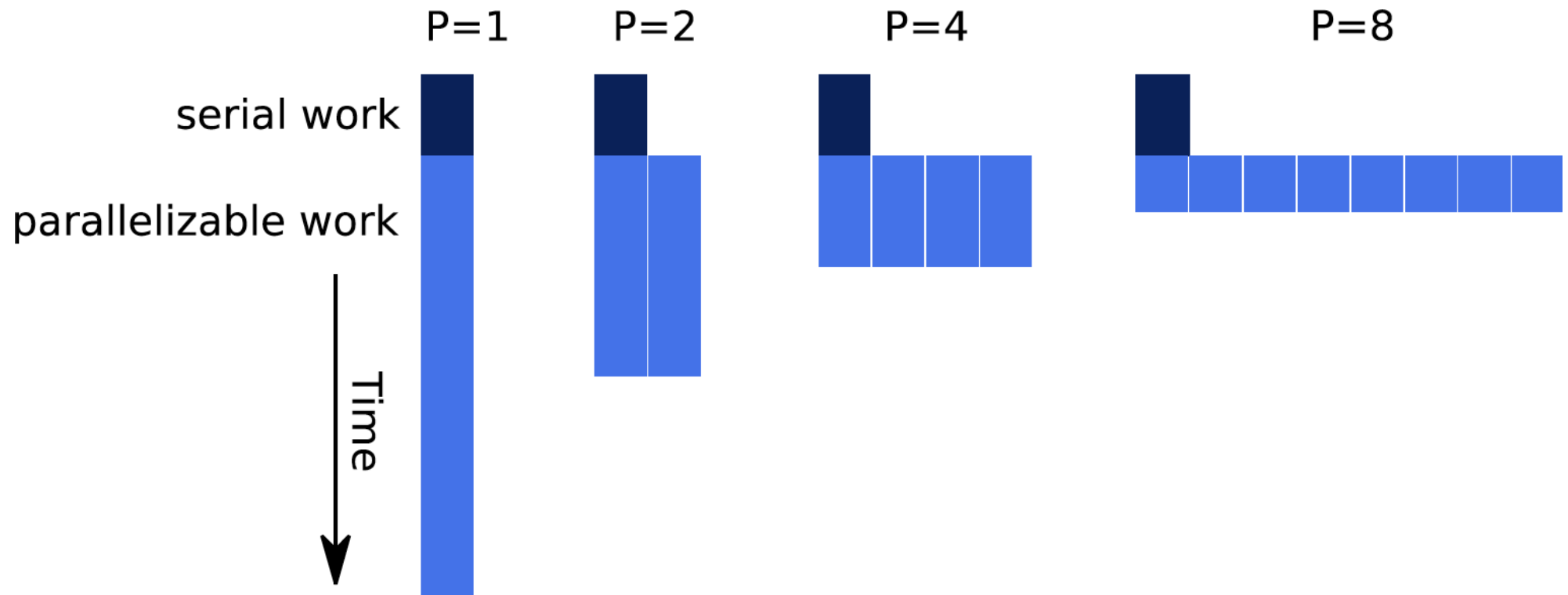
- or let f be the serial fraction of the total work,

$$\begin{aligned} W_{ser} &= fT_1 \\ W_{par} &= (1 - f)T_1 \\ S_p &\leq \frac{1}{f + (1 - f)/P} \end{aligned}$$

- **An important corollary:** Even when there are infinite resources, the speedup can not be greater than $1/f$!

$$S_\infty \leq \frac{1}{f}$$

Amdahl's Law pictorially

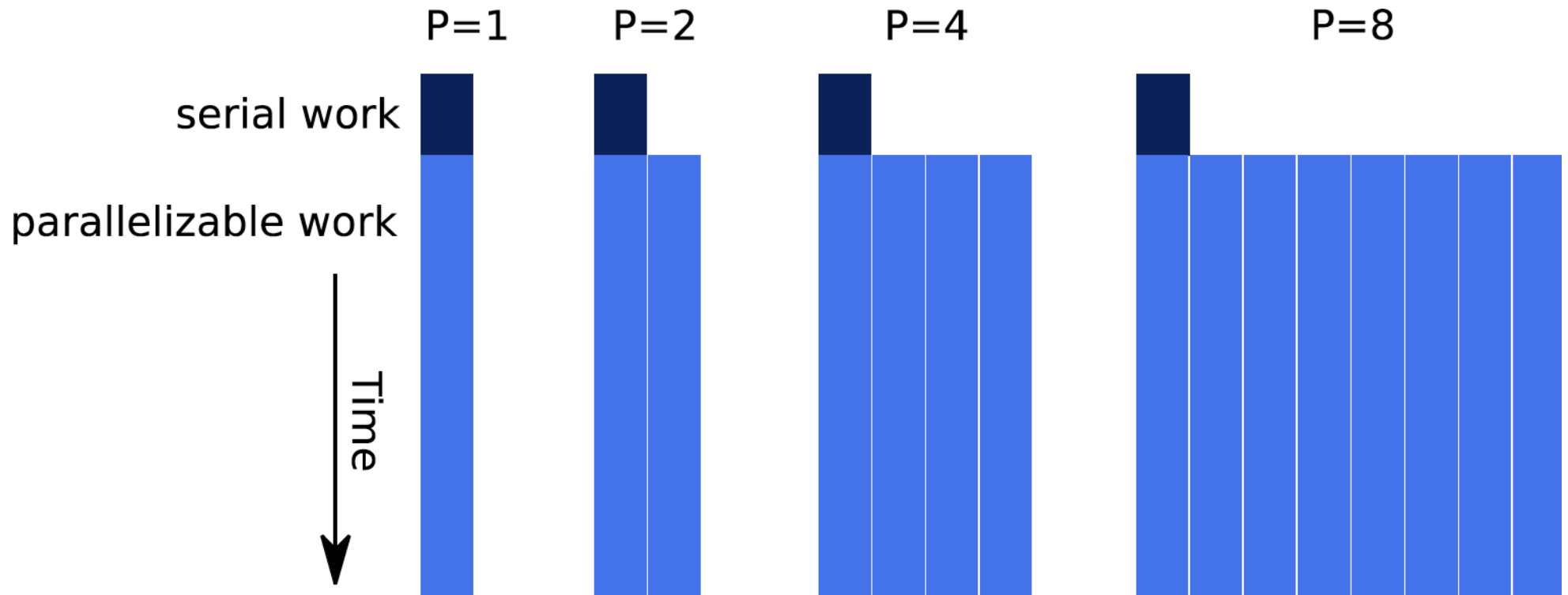


Source: McCool, Robison, Reinders

Amdahl's Law & Strong scalability

- **Amdahl's law** is closely related to a commonly used notion of scalability: **strong scaling**
- **Strong scaling:** how does the execution time vary as we **increase the number of processors** for a **fixed total problem size**?
- But, the problem sizes that we want to solve often grow as the machines that we work with grow, too!
- **Weak scaling:** how does the execution time vary as we **increase the number of processors and the total problem size at the same rate**? (attributed to Gustafson & Barsis)

Weak scaling (Gustafson--Barsis)



Source: McCool, Robison, Reinders

Modeling Performance

Modeling Performance

- At any given time, a processor can be busy with one of the following:
 - Computation
 - Data access (to memory or cache)
 - Communication
- Disk I/O (avoided in HPC except for program start/finish)
- A simple model for total execution time (excluding IO)
 - $T_{\text{total}} = \text{Flop time} + \text{Memory time} + \text{Communication time}$
 $= T_F + T_M + T_C$

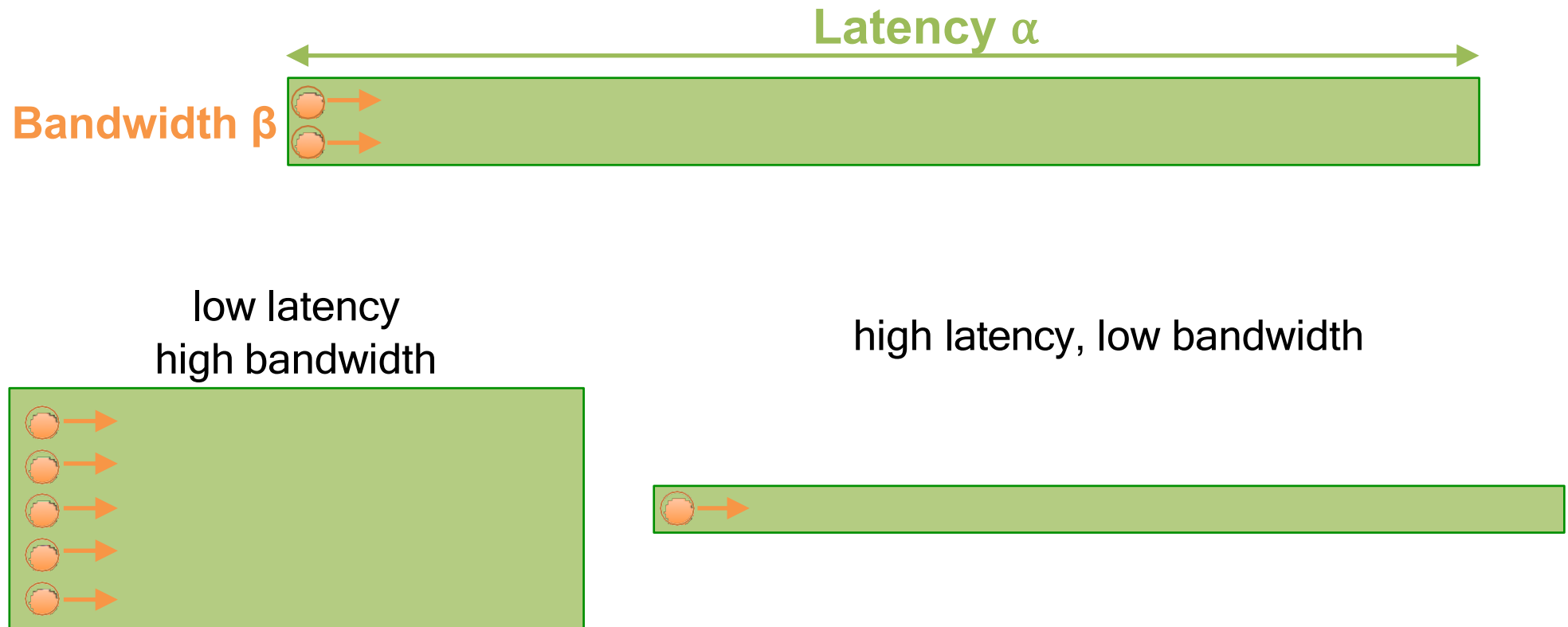
Modeling Performance

- $T_{\text{total}} = T_F + T_M + T_C$
- Definitions:
 - f = total # of floating point arithmetic (flop) operations
 - t_f = time per flop $\ll t_m$
 - m = total # of memory elements (words) moved between fast and slow memory
 - t_m = time per slow memory access
 - c = total # of bytes communicated between compute nodes
 - t_c = average time for communicating one byte
- $T_{\text{total}} = f * t_f + m * t_m + c * t_c$

Communication

- The network hardware that connects different subsystems (processing units, memory banks, compute nodes) in a system.
- **Latency (α):** Time elapsed between source & destination to transmit the first byte. Limited by the medium and length. Unit is seconds.
- **Bandwidth (β):** The rate at which data can be pumped between source and destination. Determined by the “width” of the medium. Unit is bytes/second.

Latency vs. Bandwidth



$$\text{data transmission time} = \alpha + D/\beta$$

Memory Access vs Network Communication

- Network communication:

$$\text{data transmission time} = \alpha + D/\beta$$

- Same applies for memory access (with different values)
- Typically α is much lower for memory systems
 - short distances, fast links
 - also prefetching mechanisms exist for hiding memory latency
- So t_m is generally determined by the bandwidth not the latency

Modeling performance

- Different operations can be overlapped with each other
- Prefetching:
 - overlapped computation and memory access
 - hardware/software prefetching
- Non--blocking communication primitives:
 - overlapped computation and communication
- Algorithmically overlapping computation/communication:
 - designate computation & communication threads
 - can still use blocking communication primitives

*-bound

- $T_{\text{total}} = f * t_f + m * t_m + c * t_c$
- If $T_{\text{total}} \approx c * t_c \Rightarrow$ performance is **communication-bound**
 - Performance improvements may be possible
 - Ex: graph traversals, sparse matrix computations
- If $T_{\text{total}} \approx m * t_m \Rightarrow$ performance is **memory-bound**
 - Data locality/access patterns may be improved
 - Ex: bioinformatics applications
- If $T_{\text{total}} \approx f * t_f \Rightarrow$ performance is **CPU-bound**
 - CPU performing close to its peak - as good as one can get
 - Ex: dense matrix computations

Arithmetic Intensity

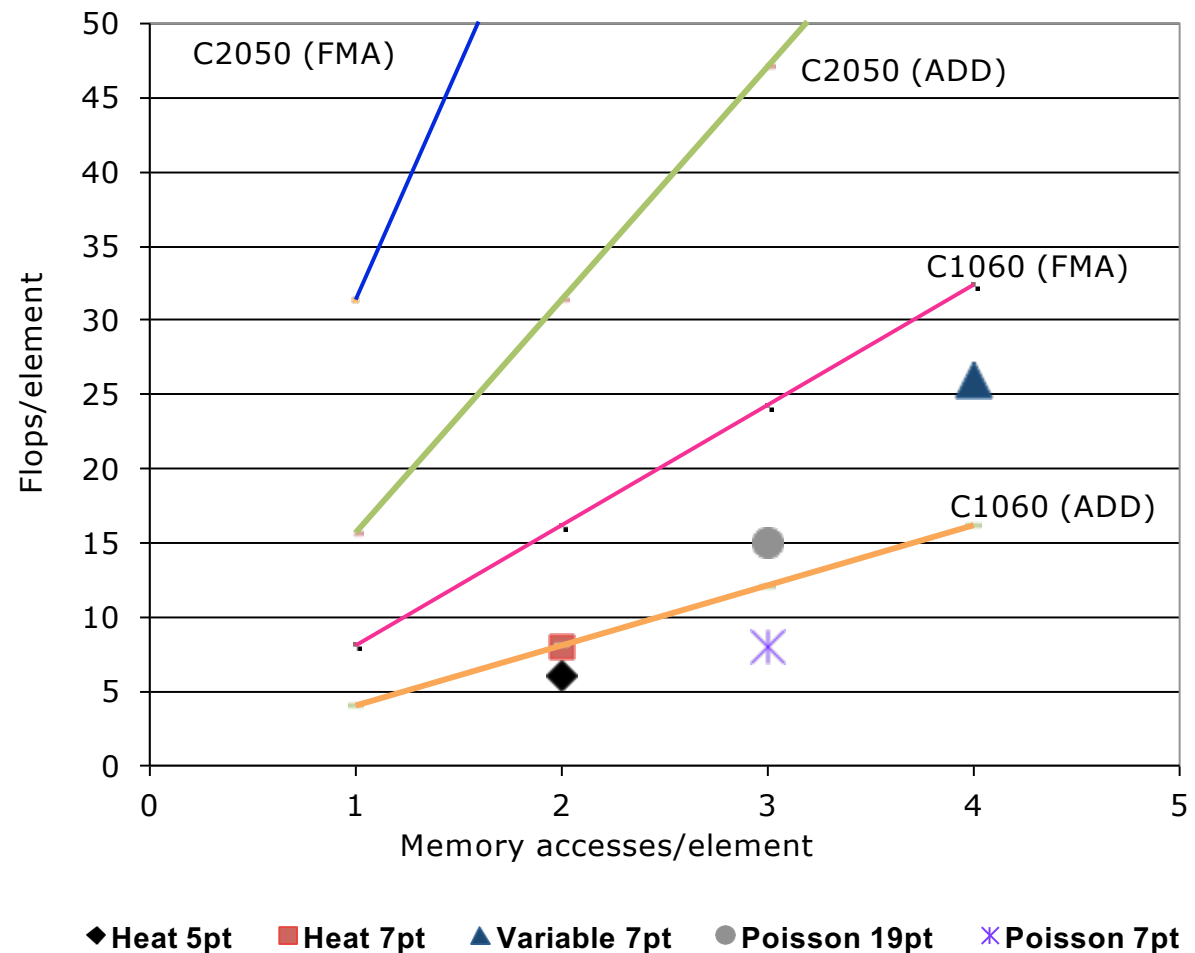
- We will discuss communication later –let's talk about computation and memory interplay for now...
- Assumptions:
 - Only 2 levels in the hierarchy:
fast (cache) and **slow** (memory)
 - All data are initially in the slow memory
- **Arithmetic Intensity: $q = f / m$**
 - average number of flops per slow memory access
 - also known as flop:byte ratio
 - key to the efficiency of an algorithm

Machine Balance

- $T_{\text{total}} = f * t_f + m * t_m = f * t_f * (1 + t_m/t_f * 1/q)$
 - t_m/t_f : machine balance -key to machine efficiency
 - Larger $q \Rightarrow$ closer to minimum possible time
 - $q \geq t_m/t_f$ needed to get at least half of peak speed
- Minimum possible time = $f * t_f$
 - achieved if $(t_m/t_f * 1/q) \approx 0$
 - if all data in fast memory
 - if the arithmetic intensity is very very high
 - **CPU-bound performance**

Application performance

- Lines indicate machine's performance
 - FMA: fused--multiply add
 - ADD: add
- Points indicates the application characteristics
- Most the applications are memory-bound in this example because they stay under the lines



■ Img courtesy-- Didem Unat

Case study: Matrix-Vector multiply

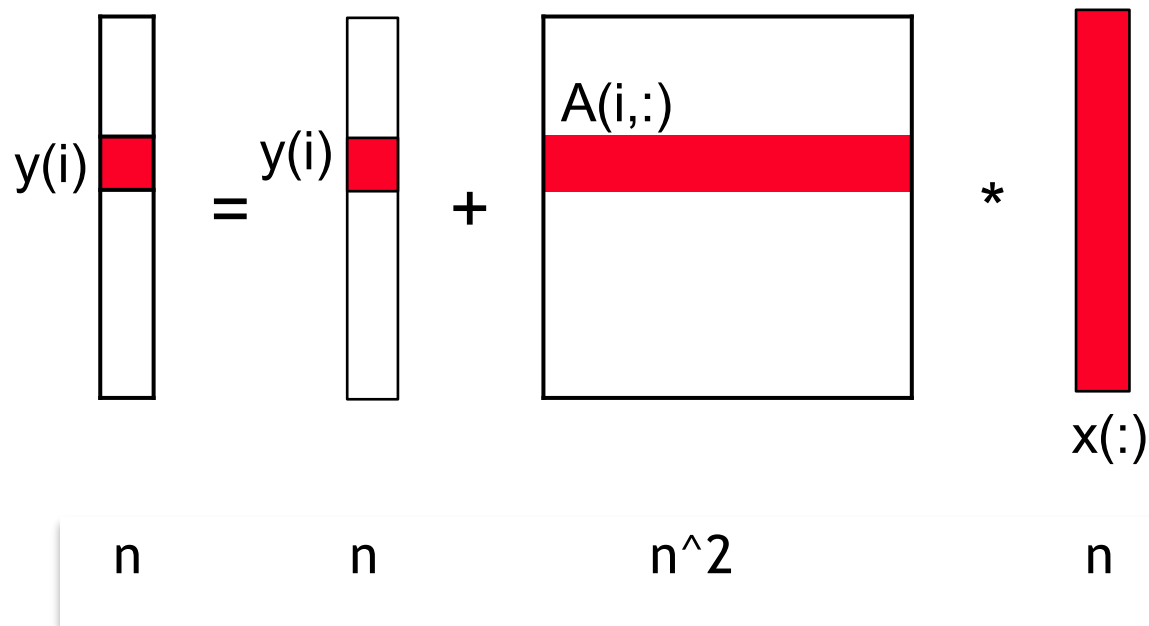
```
{implements  $y = y + A*x$ }
```

```
for i = 1:n
```

```
  for j = 1:n
```

```
     $y(i) = y(i) + A(i,j)*x(j)$ 
```

- how many bytes accessed?
- how many flops performed?



Case study: Matrix-Vector multiply

```
{read x(1:n) into fast memory}
{read y(1:n) into fast memory}
for i = 1:n
  {read row i of A into fast memory}
  for j = 1:n
    y(i) = y(i) + A(i,j)*x(j)
  {write y(1:n) back to slow memory}
```

- **m** = number of bytes accessed on slow memory = $3n + n^2$ doubles
 - **f** = number of arithmetic operations = $2n^2$
 - **q** = $f / m \approx 2 / 8$ (in double precision - double 8 bytes)
- => Matrix-vector multiplication limited by the slow memory speed,
because in general $t_f \ll t_m$

Case study: Matrix-Vector multiply

$$T_{\text{total}} = f * t_f * (1 + t_m/t_f * 1/q)$$

- Assume memory bandwidth is 20 GB/s, $q \approx 2/8 = 0.25$.
- What's the peak performance, if CPU peak is 20 Gflop/s?
- How about if CPU peak is 1 Gflop/s?

Case study: Matrix-Vector multiply

```
{read x(1:n) into fast memory}  
{read y(1:n) into fast memory}  
for i = 1:n  
  {read row i of A into fast memory}  
  for j = 1:n  
     $y(i) = y(i) + A(i,j) * x(j)$   
  {write y(1:n) back to slow memory}
```

- Note y & $A(i,:)$ are accessed in bulk to reduce latency overheads
- **Important assumptions:**
 - Fast memory is large enough to hold $3n$ doubles
 - Time to access data in fast memory is 0
- Using (Intel) intrinsics, it's possible to control what is/isn't cached
 - So one can do fine with a cache (slightly) larger than n

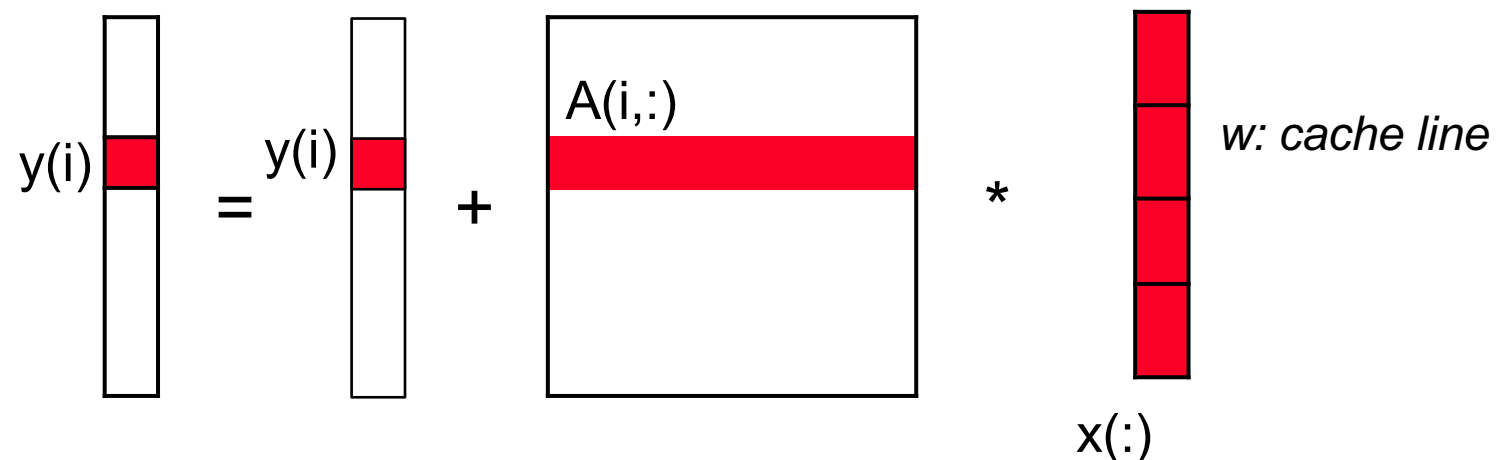
Case study: Matrix-Vector multiply

- How many cache misses would such an implementation incur on x , if the cache size was $< n$ and LRU(**Least Recently Used**) policy is used?

n^2/w w :cache-line width

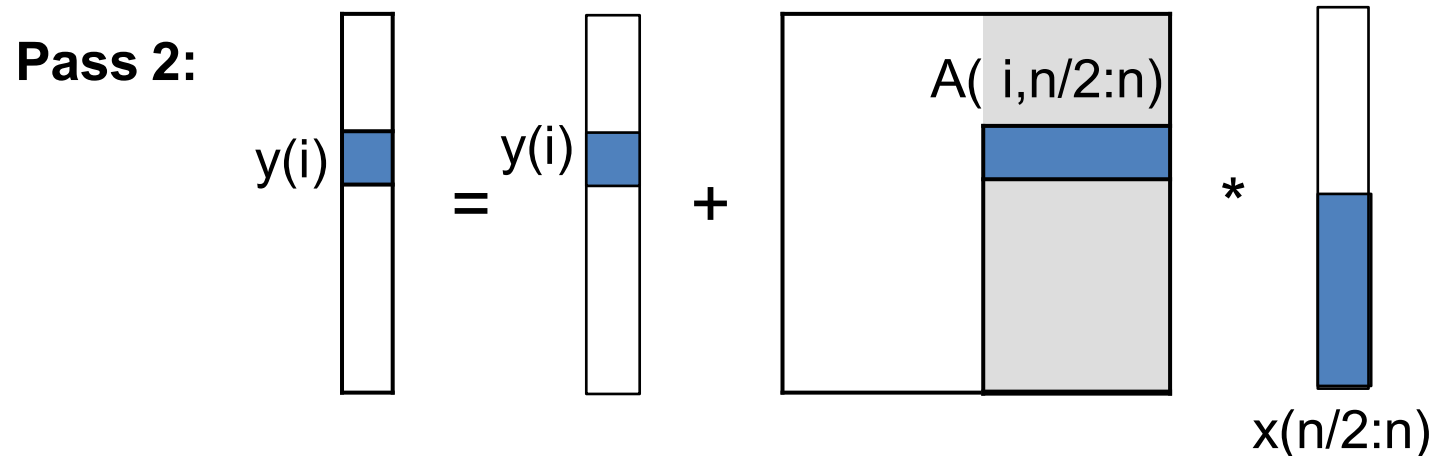
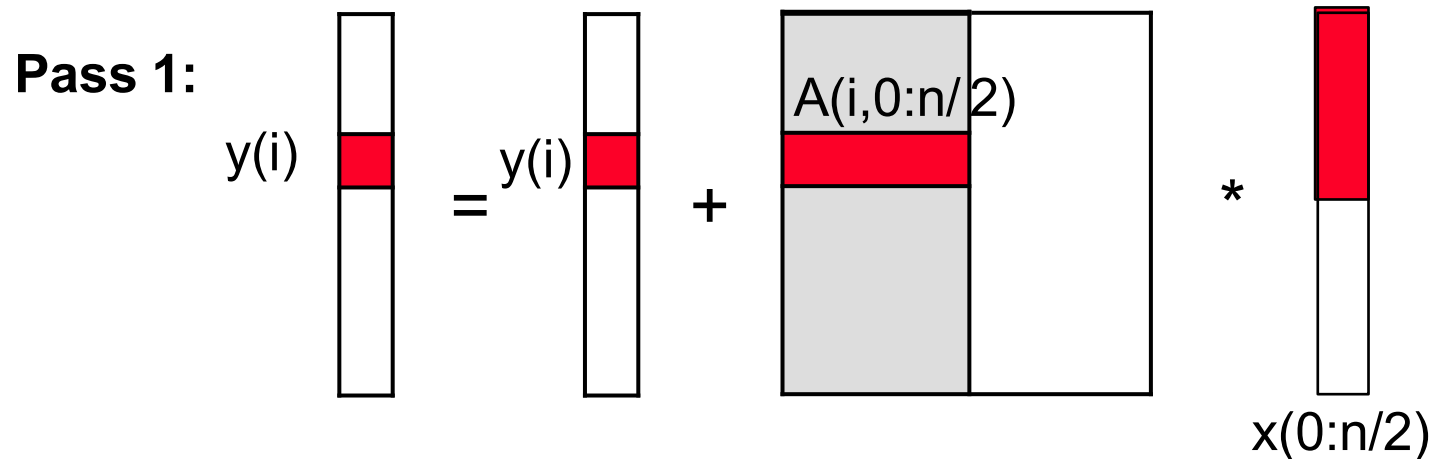
For every row of A , n/w misses occur

Is it possible to do better?



Case study: Matrix-Vector multiply

- A blocked algorithm should do better



Case study: Matrix-Vector multiply

With cache--blocking:

- read each element of **A** once from slow memory
- read each element of **x** once from slow memory
- read & write each element of **y** twice from/to memory
 - Twice because they are two blocks
- total cache misses on x: n/w
- total cache misses on y: $2n/w$

In practice though, you probably will not see any significant performance difference between the two implementations because:!

- There are multiple levels of cache to ease misses
- There are advanced prefetching hardware to hide latencies between cache levels-- accesses are very predictable in mat-vec case.

Case Study 2: Matrix multiply [1]

```
{implements C = C + A*B}
```

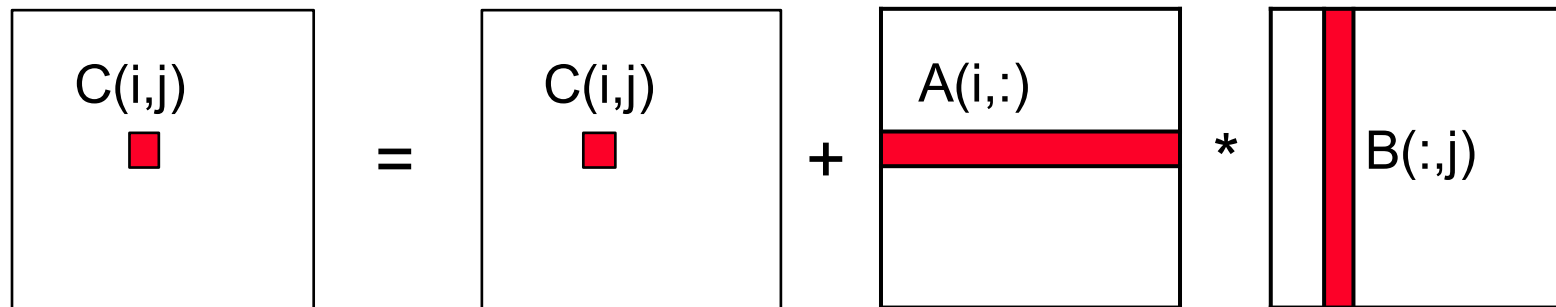
```
for i = 1 to n
```

```
  for j = 1 to n
```

```
    for k = 1 to n
```

```
      C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

- how many bytes accessed?
- how many flops performed?

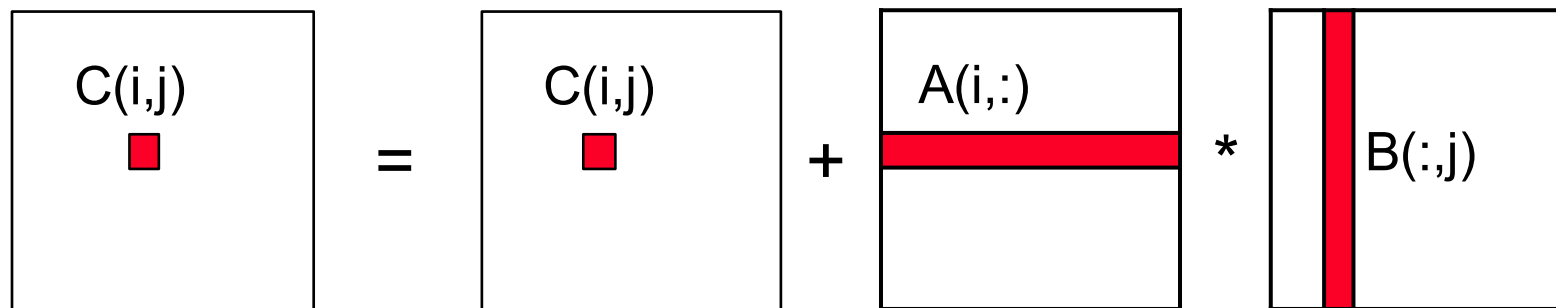


Algorithm has $2 \cdot n^3 = O(n^3)$ flops and operates on $4 \cdot n^2$ words (8 bytes for double) of memory

q can potentially be as large as $2 \cdot n^3 / (4 \cdot n^2 \cdot 8) = O(n)$

Case Study 2: Matrix multiply [1]

```
{implements  $C = C + A * B$ }  
for i = 1 to n  
  {read row i of A into fast memory}  
  for j = 1 to n  
    {read  $C(i,j)$  into fast memory}  
    {read column j of B into fast memory}  
    for k = 1 to n  
       $C(i,j) = C(i,j) + A(i,k) * B(k,j)$   
    {write  $C(i,j)$  back to slow memory}
```



A more interesting example: Matrix multiply [1]

Number of slow memory references on unblocked matrix multiply

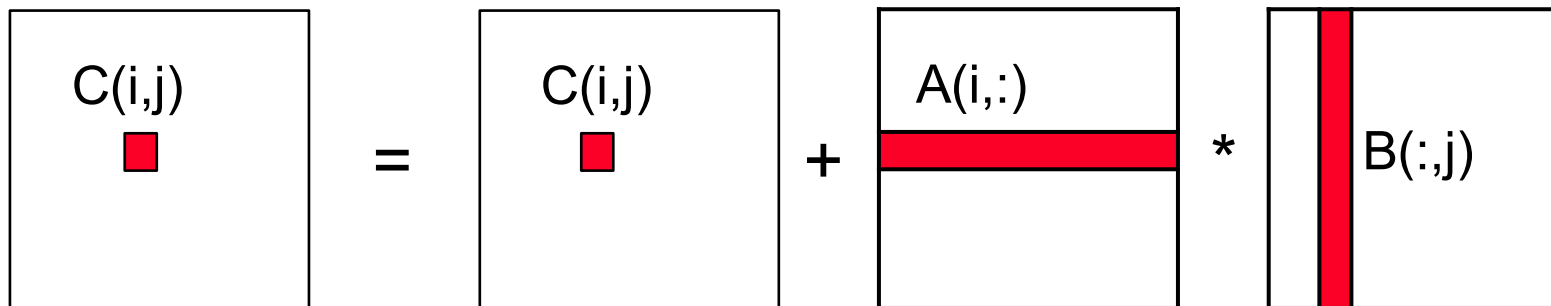
$$\begin{aligned} m = n^3 & \quad \text{to read each column of B } n \text{ times} \\ + n^2 & \quad \text{to read each row of A once} \\ + 2n^2 & \quad \text{to read and write each element of C once} \\ = n^3 + 3n^2 \end{aligned}$$

$$\text{So } q = f / m = 2n^3 / (8 * (n^3 + 3n^2))$$

$\approx 2 / 8$ for large n - NO improvement over matrix-vector multiply!

Inner two loops are just matrix-vector multiply, of row i of A times B

Similar analysis holds for any other ordering of the 3 loops



Blocked (Tiled) Matrix Multiply

Consider A, B, C to be N -by- N matrices of b -by- b subblocks where $b = n / N$ is called the *block size*

for $i = 1$ to N

for $j = 1$ to N

{read block $C(i,j)$ into fast memory}

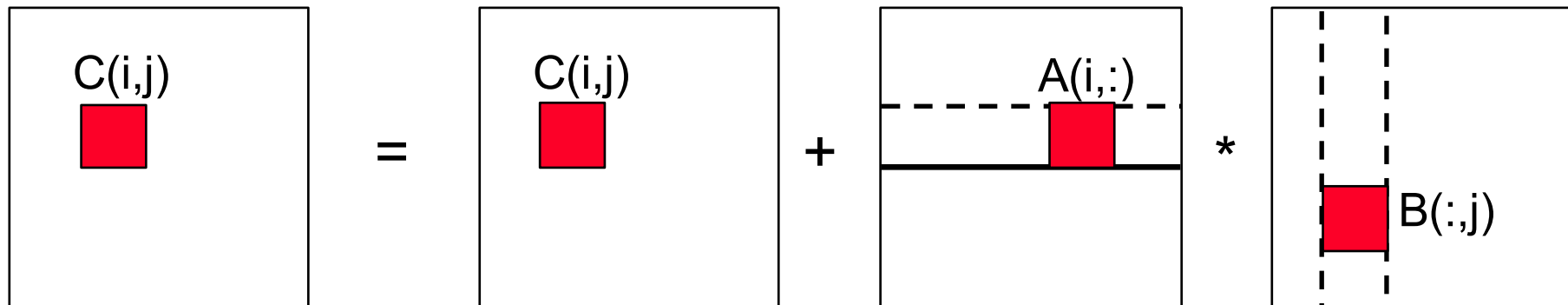
for $k = 1$ to N

{read block $A(i,k)$ into fast memory}

{read block $B(k,j)$ into fast memory}

$C(i,j) = C(i,j) + A(i,k) * B(k,j)$ {do a matrix multiply on blocks}

{write block $C(i,j)$ back to slow memory}



Blocked (Tiled) Matrix Multiply [1]

$m = N * n^2$ read each block of B N^3 times ($N^3 * b^2 = N^3 * (n/N)^2 = N * n^2$)

+ $N * n^2$ read each block of A N^3 times

+ $2n^2$ read and write each block of C once

$$= (2N + 2) * n^2$$

$f = 2n^3$ for matrix multiplication

$$\Rightarrow q = f / m = 2n^3 / ((2N + 2) * n^2) \approx n / N = b \text{ for large } n$$

- So matrix multiply ($q \approx b$) can be much faster than matrix-vector multiply ($q = 2$)

Using Analysis to Design/Understand Machines

- Performance of blocked matrix multiplication increases with b
- Limit:** All three blocks ($A_{(i,k)}$, $B_{(k,j)}$ and $C_{(i,j)}$) must fit in cache
- M_{fast} :** cache (fast memory) size
 $\Rightarrow 3b^2 \leq M_{fast}$, so $b \leq (M_{fast}/3)^{1/2}$
- Recall:** $T_{total} = f * t_f * (1 + t_m/t_f * 1/q)$
 - $q \geq t_m/t_f$ to get 1/2 of peak speed
 - Arithmetic intensity $q \approx b$ $\Rightarrow M_{fast} \geq 3b^2 \approx 3q^2 = 3(t_m/t_f)^2$
- Reasonable size for L1 cache, but not for registers

		required
	t_m/t_f	KB
Ultra 2i	24.8	14.8
Ultra 3	14	4.7
Pentium 3	6.25	0.9
Pentium3M	10	2.4
Power3	8.75	1.8
Power4	15	5.4
Itanium1	36	31.1
Itanium2	5.5	0.7

Back to matrix multiplication [1]

- The blocked algorithm changes the order in which values are accumulated into each $C[i,j]$ by applying commutativity and associativity
 - Slightly different answers from naïve code due to roundoff - OK
- For the blocked algorithm, $q \approx b \leq (M_{\text{fast}}/3)^{1/2}$, and there is a lower bound result that says we cannot do any better than this (using only associativity, so still doing n^3 multiplications - no Strassen-like algorithms)
- **Theorem (Hong & Kung, 1981):** Any reorganization of the matrix multiplication algorithm is limited to $q = O((M_{\text{fast}})^{1/2})$

Libraries:

Basic Linear Algebra Subroutines (BLAS)

- Industry standard interface (evolving) - www.netlib.org/blas
- History
 - **BLAS1 (1970s):**
 - vector operations: dot product, **saxpy** ($y = \alpha * x + y$), etc.
 - $m = 2n$, $f = 2n$, $q = f/m = \text{computational intensity} \sim 1$ or less
 - **BLAS2 (mid 1980s):**
 - matrix-vector operations: matrix vector multiply, etc.
 - $m = n^2$, $f = 2n^2$, $q \sim 2$, less overhead - somewhat faster than **BLAS1**
 - **BLAS3 (late 1980s)**
 - matrix-matrix operations: matrix matrix multiply, etc.
 - $m \leq 3n^2$, $f = O(n^3)$, so $q = f/m$ is typically very large, **BLAS3** is potentially much faster than **BLAS2**
- Good algorithms use **BLAS3** when possible

BLAS Implementations, LAPACK, ScaLAPACK libraries

- **BLAS** is an interface, vendors & others supply optimized implementations
 - **Netlib BLAS** (reference implementation)
 - **ATLAS** (Automatically Tuned Linear Algebra Subroutines)
 - Tuned during installation
 - Apple Accelerate, AMD ACKL, Cray libsci, Intel MKL, Goto BLAS, Nvidia cuBLAS, etc.
- **LAPACK** (Linear Algebra PACKage): built on top of **BLAS** for common linear algebra kernels (to solve systems of linear equations, least squares, eigenvalue and SVD problems)
- **ScaLAPACK**: distributed memory implementation of LAPACK

Readings

- Chapter 2 from Pacheco
- [1] Extra Material on Matrix--Multiply

Online Lecture by Jim Demmel CS 267: Applications of Parallel Computers at UC Berkeley

<https://www.youtube.com/watch?v=2pFOivcJ74g&list=PLEC62450380D4B27D&index=11>